

Understanding This Dockerfile

```
FROM python:3.12-alpine
WORKDIR /app
COPY requirements.txt ./
RUN pip install -r requirements.txt
COPY . .
CMD ["flask", "run", "--host=0.0.0.0"]
```

◆ Line 1: Base Image

```
FROM python:3.12-alpine
```

- Uses **Python 3.12** based on **Alpine Linux**
-  Small and secure base image
-  Includes `python` , `pip` , and common libraries

◆ Line 2: Set Working Directory

```
WORKDIR /app
```

- Sets `/app` as the **working directory** inside the container
- All future commands (e.g., `COPY` , `RUN`) will execute relative to `/app`

◆ Line 3–4: Install Dependencies

```
COPY requirements.txt ./  
RUN pip install -r requirements.txt
```

- Copies `requirements.txt` into the container
- Installs Python packages listed in it using `pip`

⚠ This is separated from `COPY . .` to take advantage of Docker cache

◆ Line 5: Copy App Files

```
COPY . .
```

- Copies all files from your current directory **into the container**
- Ensures `app.py` and other source files are inside `/app`

◆ Line 6: Start the Flask App

```
CMD ["flask", "run", "--host=0.0.0.0"]
```

- Default command when the container starts
- Runs `flask run` and listens on all interfaces (port 5000 by default)

Replace with `gunicorn` in production for better performance

✓ Summary

Line	Purpose
FROM	Start from Python Alpine base
WORKDIR	Set working directory
COPY requirements.txt	Prepare dependencies
RUN pip install	Install them
COPY . .	Bring in app code
CMD	Launch Flask server

Try it Yourself!

```
docker build -t flask-app .  
docker run -d -p 80:5000 flask-app
```

Visit <http://localhost> 